AEM vs Edge Delivery Services — when and where to use which

A decision guide for AEM as a Cloud Service: traditional AEM Sites delivery (Publish + Dispatcher), Edge Delivery Services (EDS), headless AEM, and the hybrid of both.

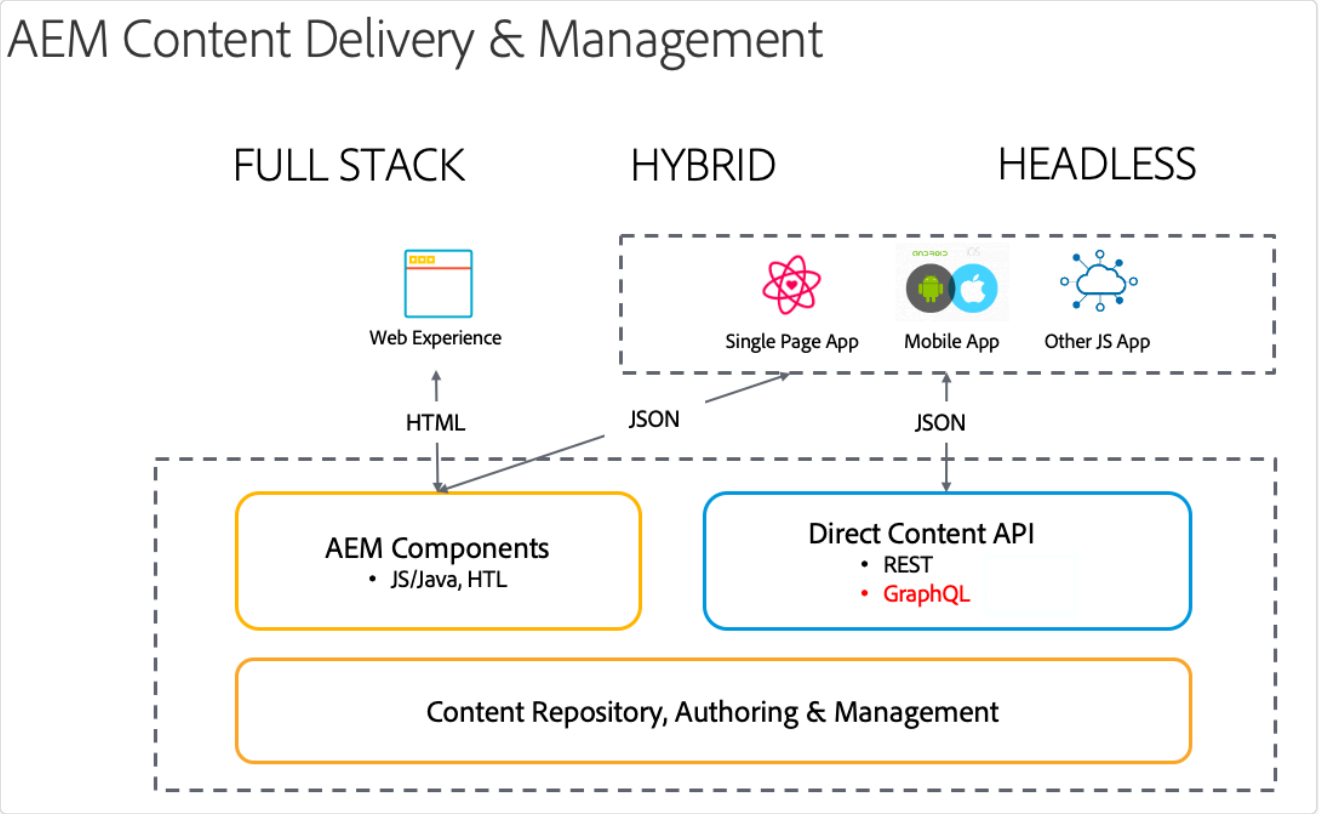
1. First, the key point: EDS *is* AEM

"AEM vs EDS" is really a choice of **how a site is delivered and built**, not a choice of product. Edge Delivery Services is part of Adobe Experience Manager, and both models can use the same **AEM Author** (Sites + Assets) for content.

> *"Edge Delivery Services is a composable set of services that allows for a high degree of flexibility in how you author content on your website. It replaces AEM Publish / Dispatcher and the traditional way of building experiences with AEM Core Components with a multi-cloud SaaS solution and pure front-end development approach."* — Adobe Experience League, Edge Delivery Services Overview

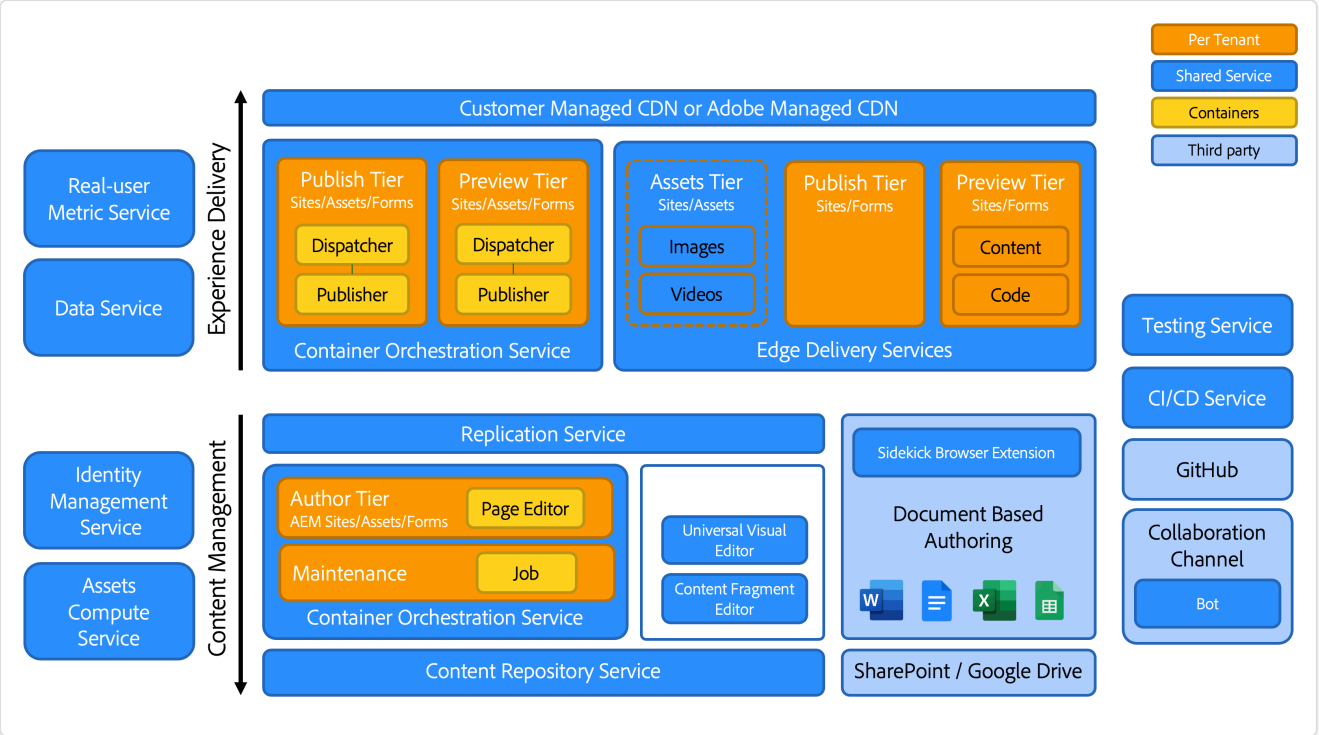
So there are three delivery models, and they can be mixed:

Model	What delivers the experience	Typical use
A. Traditional AEM Sites ("full stack")	**AEM Publish tier + Dispatcher**, pages rendered by AEM Components (HTL / Java) on the server	Complex, login-based or logic-heavy sites; existing AEM implementations
B. AEM with Edge Delivery Services	**Edge Delivery publish tier** — semantic HTML decorated with plain JS/CSS from Git	Marketing, brand, product, content and campaign sites where speed, SEO and fast delivery matter
C. AEM Headless	**APIs** (GraphQL / REST) over Content Fragments, rendered by an app	Mobile apps, single-page apps, kiosks, other channels



Source: Adobe Experience League — Introduction to AEM as a Headless CMS

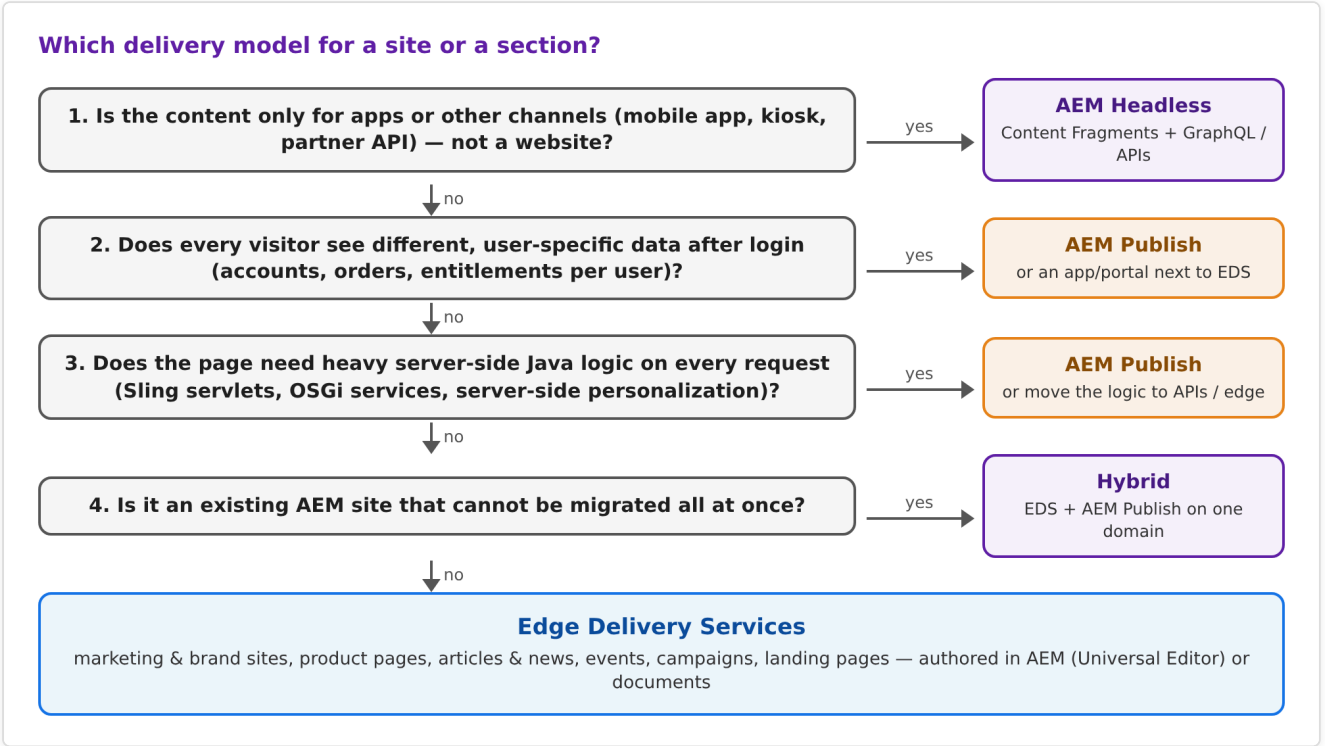
The complete AEM as a Cloud Service picture shows both delivery tiers side by side — the **AEM Publish tier** (Publisher + Dispatcher) and **Edge Delivery Services** (Publish, Preview and Assets tiers) — under one CDN, fed by the same content management layer:



Source: Adobe Experience League — Architecture Introduction (AEM as a Cloud Service)

2. Quick answer

Use Edge Delivery Services when…	Use traditional AEM Publish when…	Use AEM Headless when…
The site is public, content-driven: brand, marketing, product, articles/news, events, campaigns, landing pages	Visitors log in and see **user-specific data or permissions** (authorization per user/group)	Content must feed **apps or other channels**, not only a website
Performance, SEO/GEO and Core Web Vitals matter (EDS targets a Lighthouse score of 100)	Pages need **heavy server-side logic** on each request (Sling servlets, OSGi services)	A front-end team builds an SPA / native app
You want to **launch fast** and change code without build pipelines	You rely on Sites features not available on EDS (e.g. **ContextHub/targeting, Timewarp**)	The same structured content is reused across many channels
Authors work in **AEM (Universal Editor)** or in **Word/Google Docs**	An existing, large AEM implementation that is not migrated yet (→ hybrid)	



3. How each model works

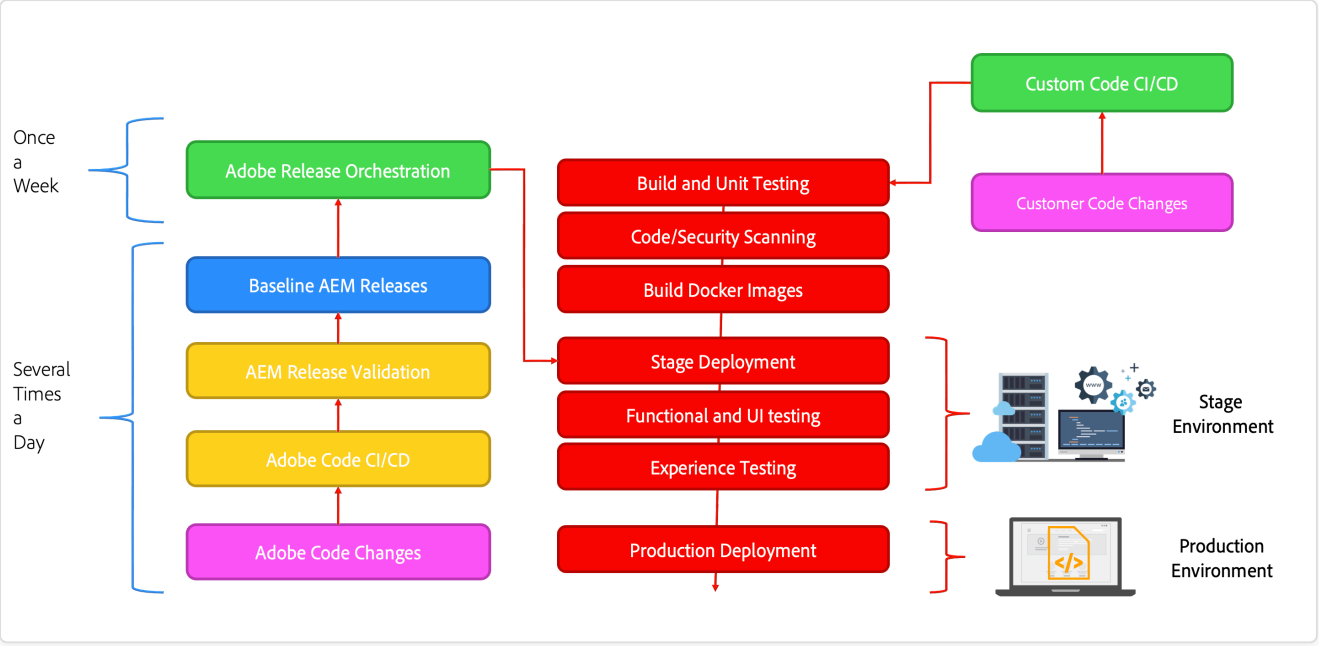
3.1 Traditional AEM Sites — Author → Publish + Dispatcher → CDN

Content is authored on the **AEM Author** tier (Page Editor or Universal Editor), **replicated** to the **Publish** tier, rendered **server-side** by AEM Components (HTL + Java/Sling), cached by the **Dispatcher** and the CDN.

> *"The AEM Publish tier: runs a farm of standard AEM publishers and dispatchers, allowing for the dynamic rendering of web pages and API content (for example, GraphQL) assembled with published content. Is primarily based on server-side application logic."*
> — Adobe Experience League, Architecture Introduction

Request flow (Experience League, *Content Delivery Flow*): browser → **CDN** → (not cached) **Dispatcher** → (not cached) **AEM Publish**; HTML is cached for 300 seconds by default at the Dispatcher, which the CDN respects.

Code is a **Maven full-stack project** (Java, OSGi configuration, HTL, front-end module) stored in a Cloud Manager repository and deployed through **Cloud Manager pipelines** (build, code quality, tests, deploy) to author, preview and publish:

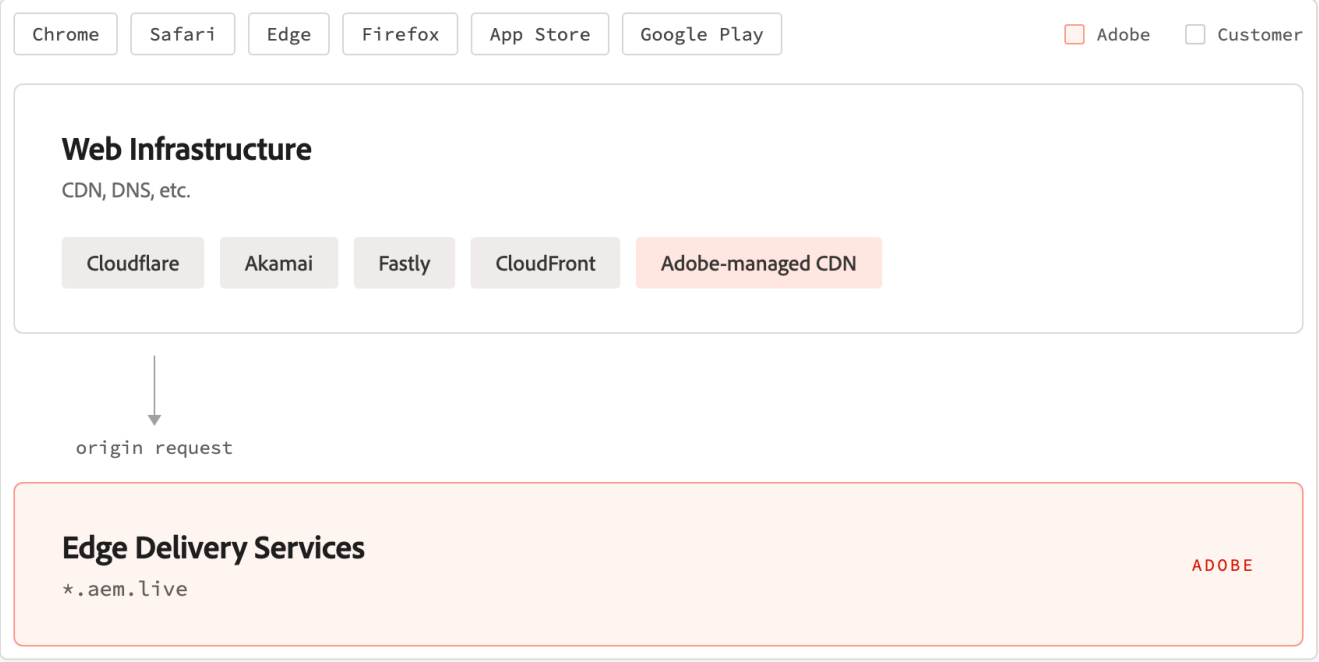


Source: Adobe Experience League — Architecture Introduction

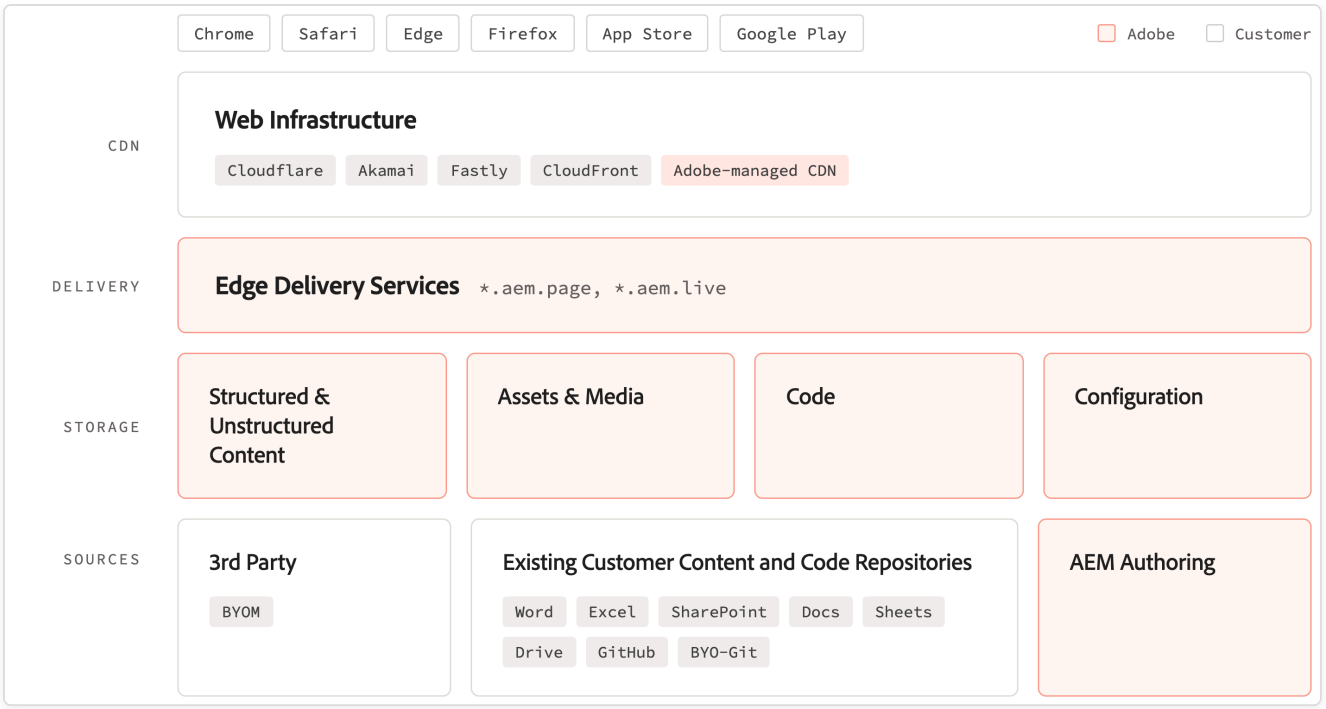
3.2 AEM with Edge Delivery Services — Author/Documents → EDS → CDN

Content is authored in **AEM with the Universal Editor** (or in Word/Excel, Google Docs/Sheets, Document Authoring). On **Preview/Publish** the content is converted into **semantic HTML** and stored in the EDS content bus; the **page is decorated in the browser** by the project's blocks — plain **JavaScript and CSS from a Git repository**. Publishing **push-invalidates** the CDN cache immediately.

> *"The Edge Delivery Publish tier: allows for the dynamic rendering of web pages and API content from various content sources such as the AEM Author tier or the Document-based Authoring tier. Is based on client-side application logic and designed for maximum performance."* — Adobe Experience League, Architecture Introduction

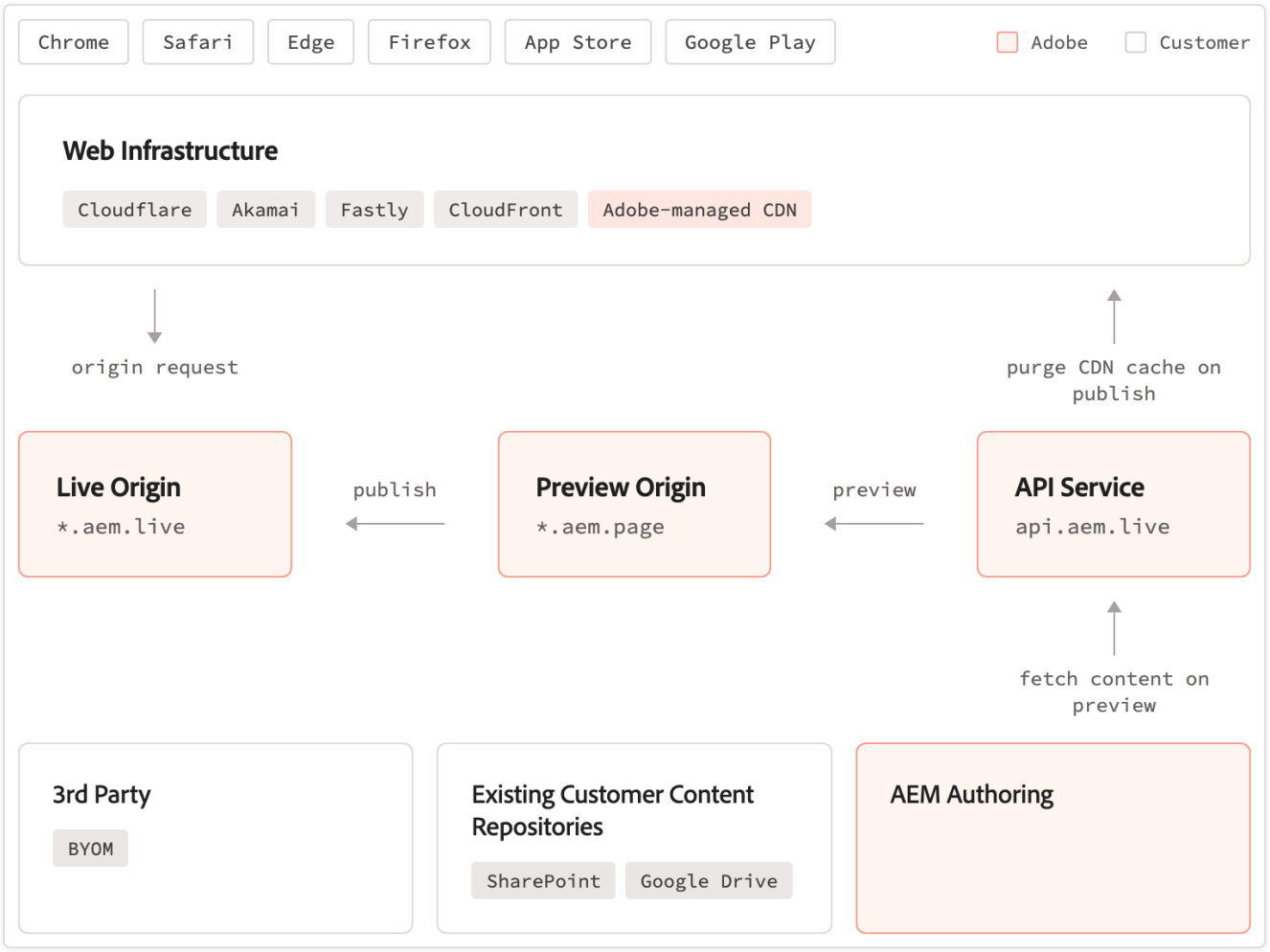


Source: aem.live — Architecture



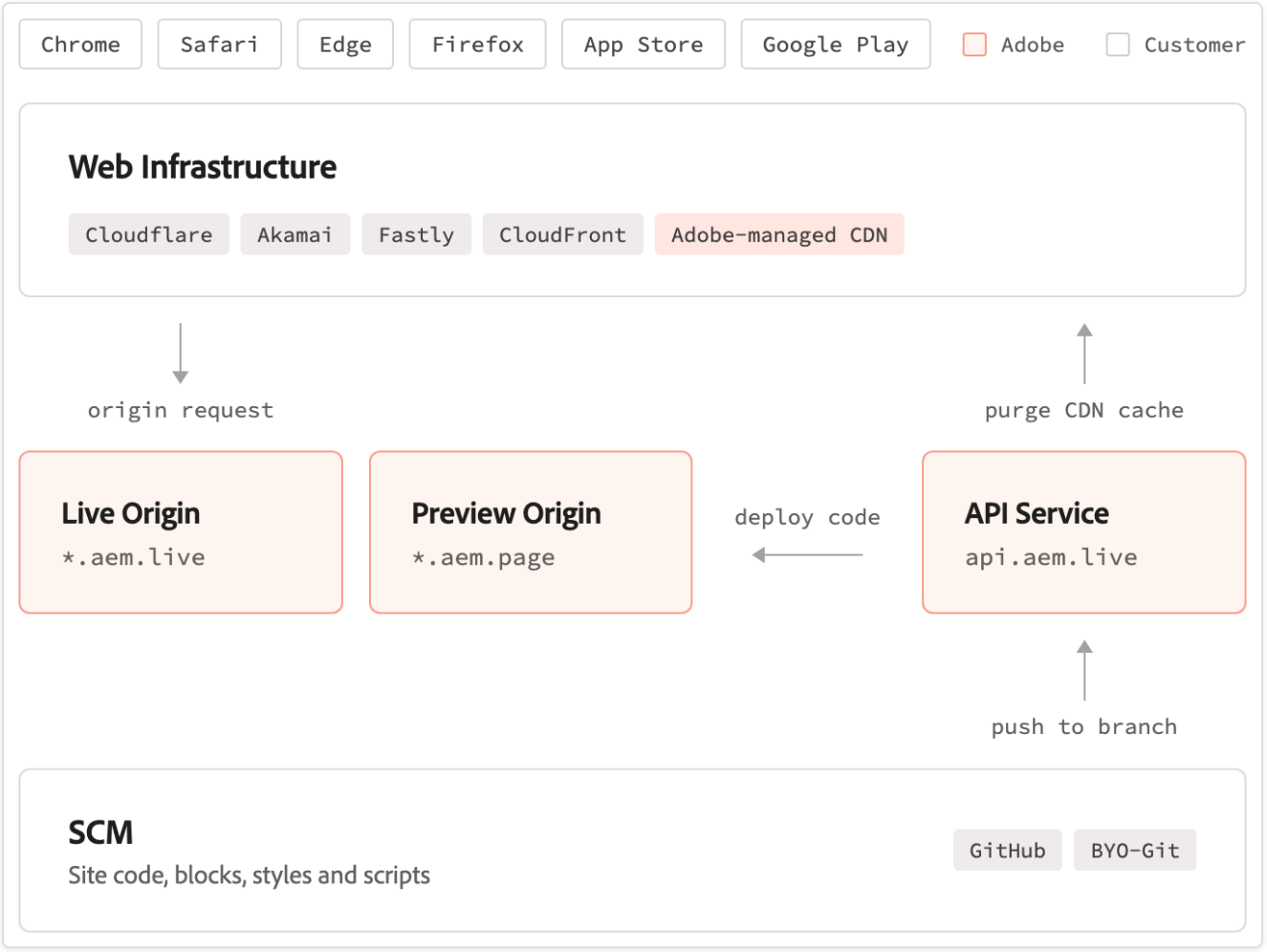
Source: aem.live — Architecture

Publishing (preview → live, with instant CDN invalidation):



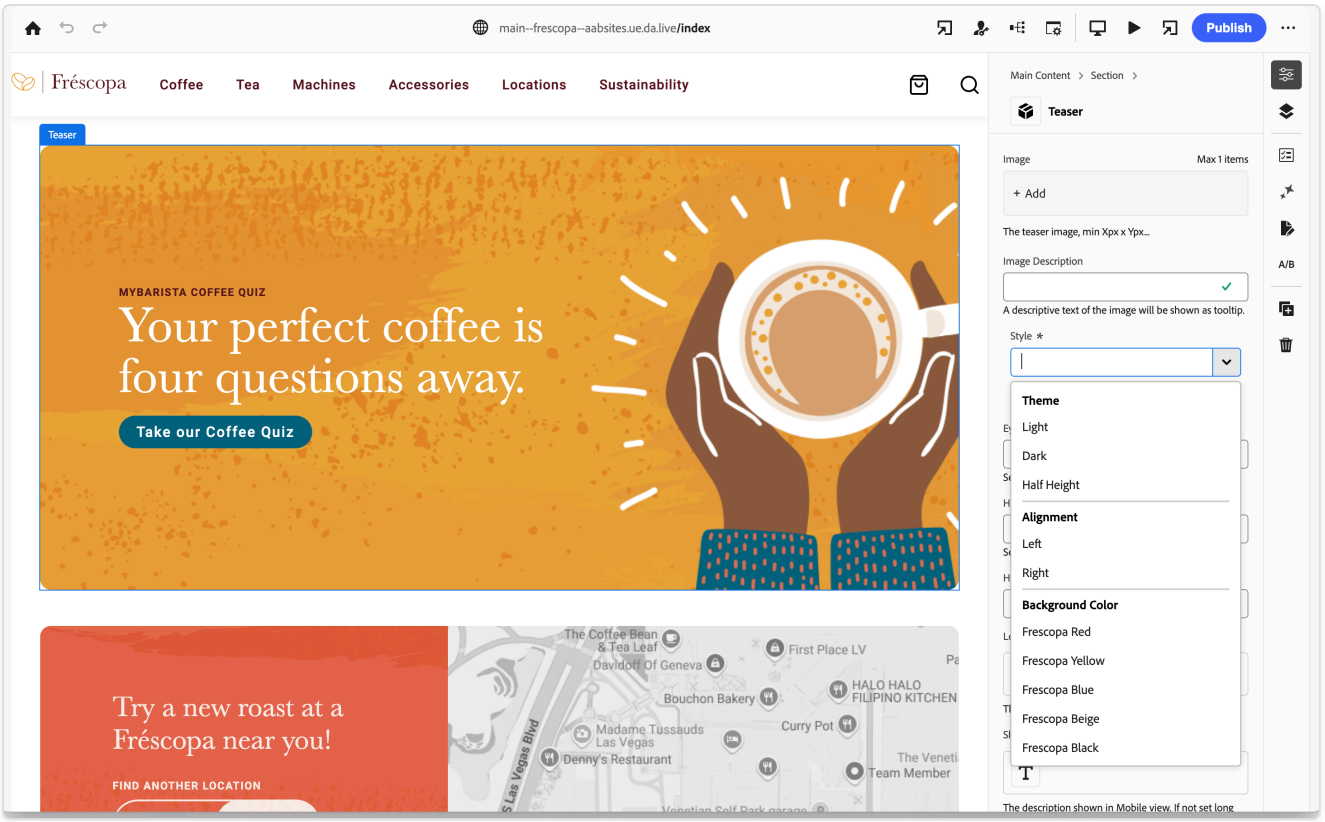
Source: aem.live — Architecture

Code: synced from Git on every push — every branch gets its own preview URL (`https://<branch>--<site>--<org>.aem.page`), no build step and no deployment pipeline:



Source: aem.live — Architecture

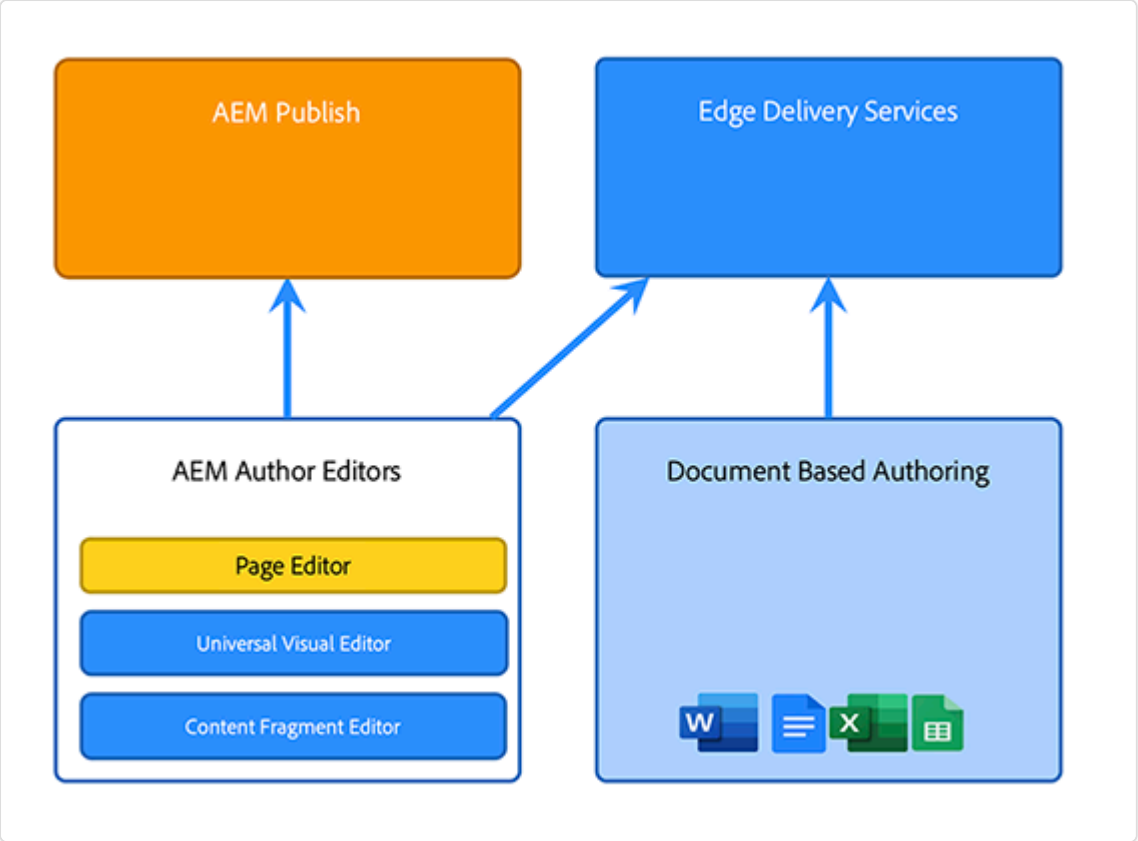
Authoring in AEM with the Universal Editor (WYSIWYG, in context, with AEM workflows, governance, MSM, Assets):



Source: Adobe Experience League — Edge Delivery Services Overview

3.3 Both together

AEM Author can publish to **both** delivery tiers — Page Editor content to AEM Publish, Universal Editor content to Edge Delivery Services — while document-based content goes only to Edge Delivery:



Source: Adobe Experience League — Architecture Introduction

---

4. Detailed comparison

Topic	Traditional AEM Sites (Publish + Dispatcher)	AEM with Edge Delivery Services
Rendering	Server-side on AEM Publish (HTL, Sling, Java)	Semantic HTML from the content source, decorated client-side by blocks (vanilla JS/CSS)
Authoring	AEM Page Editor (or Universal Editor), Sites console	AEM **Universal Editor** + Sites console, or **document-based** (Word/Excel, Google Docs/Sheets, Document Authoring)
Code	Maven full-stack project: Java, OSGi, HTL, clientlibs	Git repository with `blocks/`, `scripts/`, `styles/` — plain HTML, CSS, JavaScript, no transpiling

Topic	Traditional AEM Sites (Publish + Dispatcher)	AEM with Edge Delivery Services
Deploying code	Cloud Manager pipeline (build, quality gates, tests, deploy)	Push to Git → live on the branch URL; `main` = production code
Environments	Dev / Stage / Prod environments per program	One URL per branch (`*.aem.page` preview, `*.aem.live` live) + config per site
Caching	Dispatcher + CDN with TTLs (HTML 5 min by default)	CDN with push invalidation on publish
Performance	Depends on implementation	Built for Lighthouse 100 / Core Web Vitals; three-phase loading (eager, lazy, delayed)
Publish tier	Required	Not required when Edge Delivery handles delivery (Flexible Publish Tier, Beta)
Server-side logic	Sling servlets, OSGi services, workflows on publish	No server-side Java on delivery; use APIs, edge workers, integrations, JSON2HTML / BYOM
Visitor login	Closed user groups, per-user/group permissions	Site authentication for the whole site only — *"Only authentication is supported. Authorization is not supported."* (aem.live)
Personalization	ContextHub, Adobe Target	Adobe Target / AEP Web SDK integration and native experimentation; ContextHub/targeting **not available**
Sites features	All	Most available — MSM, language copies, Launches available; page templates partially; ContextHub, Timewarp, associated content not available; Experience Fragments → use fragments (aem.live *Feature Lifecycle*)
Forms	AEM Forms (adaptive forms)	Edge Delivery Services for AEM Forms (AEM Forms license required)
Commerce	Commerce integration framework	Adobe Commerce Storefront on Edge Delivery Services

Topic	Traditional AEM Sites (Publish + Dispatcher)	AEM with Edge Delivery Services
Assets	AEM Assets	AEM Assets (Edge Delivery Assets tier)
Skills needed	AEM Java/HTL/OSGi developers	Front-end web developers (HTML/CSS/JS)

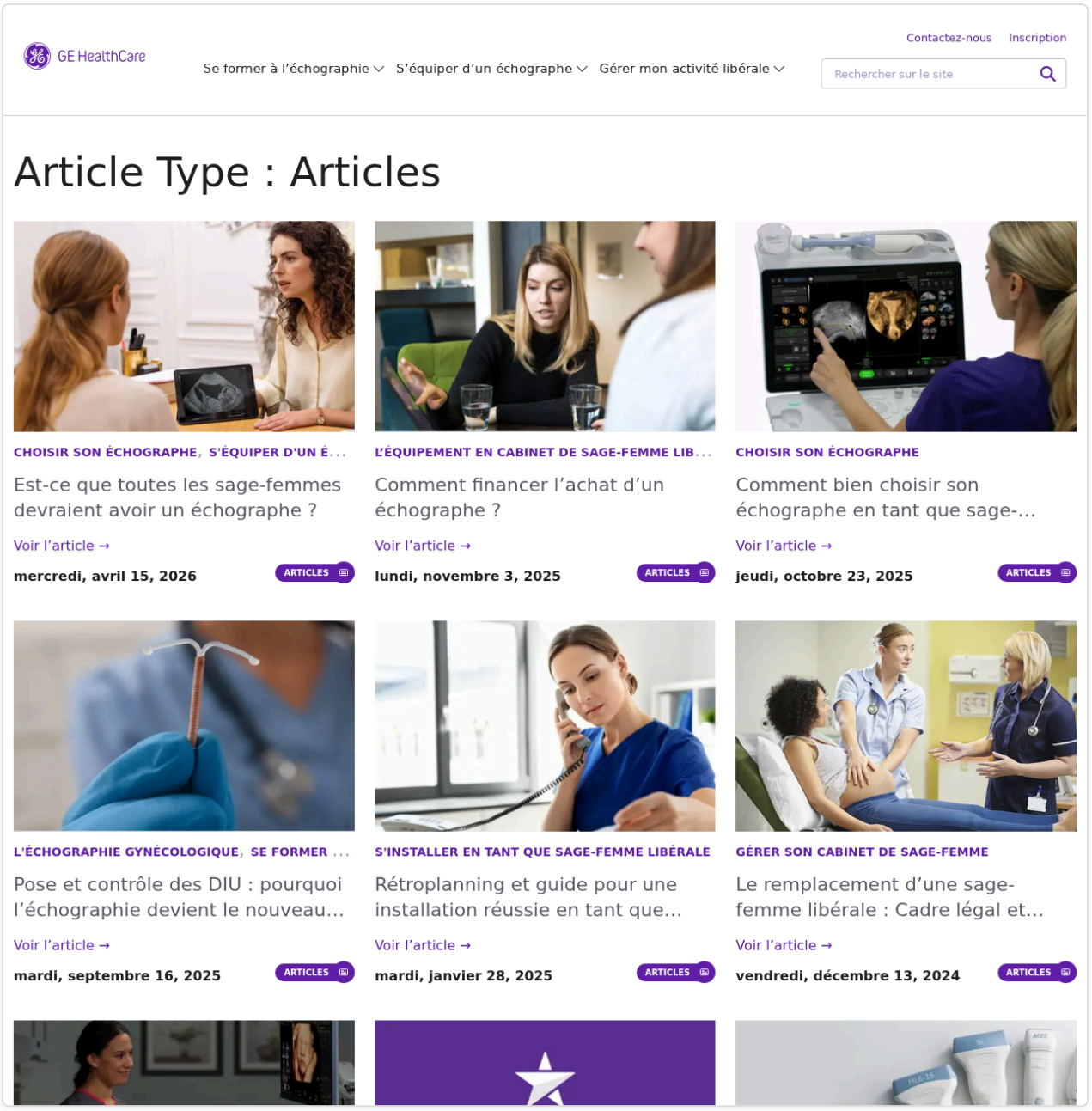
---

5. When to use Edge Delivery Services — with examples

Choose EDS for **public, content-driven websites** where speed of delivery and of development matters:

- **Brand and marketing sites** — e.g. a country site like the French midwives site migrated in this project (`/fr-fr/`), authored in AEM with the Universal Editor and delivered by EDS.
- **Product and solution pages** — product overviews, specifications, configurator entry pages.
- **Articles, news, insights, press releases** — page templates + metadata + query-index listings.
- **Events and webinars** — event pages, agendas, registration (embedded forms such as Marketo, or EDS forms).
- **Campaigns and landing pages** — fast to create, fast to load, easy to test (experimentation).
- **Multi-site / multi-language rollouts** — one code base for many sites (repoless), MSM and language copies in AEM.
- **High-traffic pages** of an existing AEM site — moved to EDS first (see hybrid).

Example — an article listing page on Edge Delivery Services from this migration (blocks: header, article cards, footer):



Why EDS here: content is public, the same for every visitor, SEO-relevant, and changes often; developers can ship block changes per branch without pipelines; authors keep AEM (Universal Editor, Assets, MSM, workflows).

6. When to keep (or choose) traditional AEM Publish — with examples

Keep AEM Publish for the parts that need what EDS does not do:

- **Authenticated portals with authorization** — customer, partner or employee portals where users see different content or permissions (*my account*, orders, entitlements, gated documents per user group).

- **Heavy server-side business logic** on every request — custom Sling servlets, OSGi services, server-side integrations that cannot move to APIs or the edge.
- **Server-side personalization with ContextHub**, or features such as **Timewarp** / **associated content** that are not available on EDS.
- **Large existing AEM implementations** with many complex components — migrate **step by step** instead of all at once.
- **Experiences built on AEM's SPA Editor / AEM Components** that the team is not ready to rebuild.

7. When to use AEM Headless

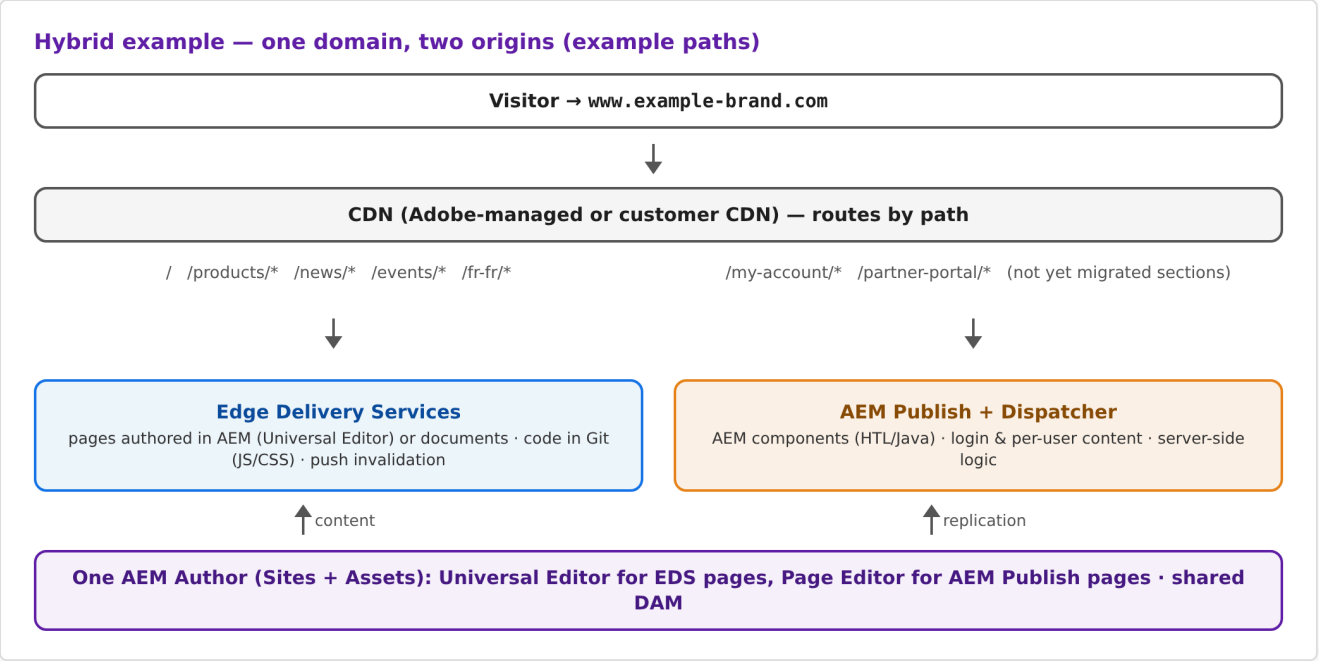
- **Mobile apps, kiosks, in-product UIs, partner feeds** — structured **Content Fragments** delivered via **GraphQL / REST**.
- A front-end team builds a **single-page app** with its own framework.
- The same content (e.g. product data, press releases) must be reused across **website and apps** — Content Fragments can also be **published to Edge Delivery Services as HTML** pages, so one fragment can serve both the site and the apps.

8. Hybrid — both on one domain

You do not have to choose for the whole domain. Adobe documents that EDS and AEM Publish can run side by side:

> *"Edge Delivery, AEM Sites and AEM Assets can co-exist on the same domain. This is a common use case for larger websites. For instance, a customer might want to migrate a particular page with high traffic to Edge Delivery Services, while all other pages might remain on the AEM Publish Tier."* — Adobe Experience League, Architecture Introduction

Cloud Manager supports this by **setting up a proxy from an existing AEM as a Cloud Service environment** to Edge Delivery Services (*"typically used when you already have an existing environment and you want to migrate part of a site to Edge Delivery Services"*). The CDN routes paths to the right origin:



Typical migration path:

1. Move the **public, high-traffic, content-driven** sections to EDS first (home, products, news, events, country sites).
2. Keep **login / portal / logic-heavy** sections on AEM Publish.
3. Use **one AEM Author** and one DAM for both; route by path on the CDN.
4. Migrate the remaining sections when their requirements can be met on EDS (or move their logic into APIs).

---

9. Summary

- **Default for new public websites and content-driven sections: Edge Delivery Services**, authored in AEM with the Universal Editor (or documents).
- **Traditional AEM Publish** for authorization-based portals, heavy server-side logic, features EDS does not offer, and not-yet-migrated parts of existing sites.
- **AEM Headless** for apps and multi-channel structured content.
- **Hybrid** is normal: one AEM Author, one domain, the CDN routing each path to EDS or AEM Publish.

References

- Edge Delivery Services Overview — https://experienceleague.adobe.com/en/docs/experience-manager-cloud-

service/content/edge-delivery/overview

- AEM as a Cloud Service architecture (Publish tier, Edge Delivery tier, co-existence on one domain, deployment architecture) — https://experienceleague.adobe.com/en/docs/experience-manager-cloud-service/content/overview/architecture
- Content Delivery Flow (CDN → Dispatcher → Publish) — https://experienceleague.adobe.com/en/docs/experience-manager-cloud-service/content/implementing/content-delivery/overview
- Introduction to Edge Delivery Services in Cloud Manager (proxy from an existing environment, flexible publish tier) — https://experienceleague.adobe.com/en/docs/experience-manager-cloud-service/content/implementing/using-cloud-manager/edge-delivery-sites/introduction-to-edge-delivery-services
- Introduction to AEM as a Headless CMS — https://experienceleague.adobe.com/en/docs/experience-manager-cloud-service/content/headless/introduction
- aem.live — Architecture — https://www.aem.live/docs/architecture
- aem.live — Where to author your site — https://www.aem.live/docs/authoring-guide
- aem.live — Authoring with AEM Sites for Edge Delivery Services — https://www.aem.live/docs/aem-authoring
- aem.live — Feature Lifecycle (Sites features available on EDS) — https://www.aem.live/docs/lifecycle
- aem.live — Keeping it 100 (performance) — https://www.aem.live/developer/keeping-it-100
- aem.live — Site authentication — https://www.aem.live/docs/authentication-setup-site
- aem.live — Adobe Target integration — https://www.aem.live/developer/target-integration
- aem.live — Forms — https://www.aem.live/developer/forms
- aem.live — Commerce Storefront — https://www.aem.live/docs/storefront
- aem.live — Publishing AEM Content Fragments to Edge Delivery Services — https://www.aem.live/developer/content-fragment-overlay